

TEK4090 – Introduction to Modern Control

Assignment 2 v4.0

Due: October 23 2024, 13:15

1. (10 points) Consider the continuous-time system,

$$\ddot{x} = u \tag{1}$$

$$J(t_0) = \frac{1}{2}x^2(0) + r \int_0^2 u^2 dt. \tag{2}$$

Hint: read §6.3 in Lewis, Syrmos, & Vraibie for some relevant example problems

- (a) Write the Hamilton-Jacobi-Bellman (HJB equation), eliminating $u(t)$

Ans: We begin by writing the hamiltonion equation:

$$H(x, u, \lambda, t) = ru^2 + \lambda^T f(x, u, t)$$

Where $\lambda \in \mathbb{R}^2$. We define f as a vector in $\mathbb{R}^2$ as well where

$$f = \begin{bmatrix} x_2 \\ u \end{bmatrix}$$

if we take $\dot{x} = x_2$ and $\dot{x}_2 = u$. We not get that the Hamiltonion is

$$H = ru^2 + \lambda_1 x_2 + \lambda_2 u$$

Our goal is to find the Hamilton Jacobi equation,

$$-J_t^* = \min_u (ru^2 + \lambda_1 x_2 + \lambda_2 u)$$

which we find by optimizing H for u and substituting the optimal u in H. First note that $\frac{\partial^2 H}{\partial u^2} = 2r \geq 0$, which is a condition for global convexity, assuming $r \geq 0$. This means that the optimal u^* we find is the global minimum of the Hamiltonion.

$$\begin{aligned} \frac{\partial H}{\partial u} &= 0 \\ &= 2ru + \lambda_2 \\ u^* &= -\frac{\lambda_2}{2r} \end{aligned}$$

We now substitute optimal control into h

$$\begin{aligned} H^* &= r \left(-\frac{\lambda_2}{2r} \right)^2 + \lambda_1 x_2 - \frac{\lambda_2^2}{2r} \\ &= \frac{\lambda_2}{4r} + \lambda_1 x_2 - \frac{\lambda_2^2}{2r} \\ &= \frac{-\lambda_2^2}{4r} + \lambda_1 x_2 \end{aligned}$$

Substituting this into J^* we get that

$$-J_t^* = \min_u (ru^2 + \lambda_1 x_2 + \lambda_2 u) = \frac{-\lambda_2^2}{4r} + \lambda_1 x_2$$

(b) Assume that the optimal cost-to-go function $J^*(t)$ is given by,

$$J^*(t) = \frac{1}{2}s_1(t)x^2(t) + s_2(t)x(t)\dot{x}(t) + \frac{1}{2}s_3(t)\dot{x}^2(t). \quad (3)$$

Use the HJB equation to derive coupled differential equations for these $s_i(t)$'s, and find boundary conditions for these equations.

Ans: We find that

- $\lambda_1 = \frac{\partial J^*}{\partial x} = s_1 x + s_2 \dot{x}$
- $\lambda_2 = \frac{\partial J^*}{\partial \dot{x}} = s_2 x + s_3 \dot{x}$
- $\frac{\partial J^*}{\partial t} = \frac{1}{2}s_1 \dot{x}^2 + s_2 x \ddot{x} + \frac{1}{2}s_2 \dot{x}^2$

If we plug this into the HJB equation we found from a), we get

$$\begin{aligned} - \left[\frac{\partial J^*}{\partial t} = \frac{1}{2}\dot{s}_1 x^2 + \dot{s}_2 s_2 x \dot{x} + \frac{1}{2}\dot{s}_3 \dot{x}^2 \right] &= \frac{-1}{4r} (s_2 x + s_3 \dot{x})^2 + (s_1 x + s_2 \dot{x}) \dot{x} \\ &= \frac{-1}{4r} (s_2^2 x^2 + 2s_2 s_3 x \dot{x} + s_3^2 \dot{x}^2) + s_1 x \dot{x} + s_2 \dot{x}^2 \\ &= \frac{-s_2^2 x^2}{4r} + x \dot{x} \left(s_1 - \frac{s_2 s_3}{2r} \right) + \dot{x}^2 \left(s_2 - \frac{s_3^2}{4r} \right) \end{aligned}$$

Now we can match the coefficients with the original J^* and get

$$-\frac{1}{2}\dot{s}_1 x^2 - \dot{s}_2 x \dot{x} - \frac{1}{2}\dot{s}_3 \dot{x}^2 = \frac{-s_2^2 x^2}{4r} + x \dot{x} \left(s_1 - \frac{s_2 s_3}{2r} \right) + \dot{x}^2 \left(s_2 - \frac{s_3^2}{4r} \right)$$

- $-\frac{1}{2}\dot{s}_1 = -\frac{s_2^2}{4r}$
- $-\dot{s}_2 = s_1 - \frac{s_2 s_3}{2r}$
- $-\frac{1}{2}\dot{s}_3 = s_2 - \frac{s_3^2}{4r}$

Which makes

- $\dot{s}_1 = \frac{s_2^2}{2r}$
- $\dot{s}_2 = \frac{s_2 s_3}{2r} - s_1$
- $\dot{s}_3 = \frac{s_3^2}{2r} - 2s_2$

Now for the initial conditions. At $t = 2 = T$, cost to go is $J^*(T) = \frac{1}{2}x(T)^2$. From $J^* = \frac{1}{2}s_1(T)x^2 + s_2(T)x\dot{x} + s_3(T)\dot{x}^2$, we draw that

- $s_1(T) = 1$
- $s_2(T) = 0$
- $s_3(T) = 0$

(c) Write the linear state feedback optimal control policy in terms of the $s_i(t)$'s.

Ans: We have $u^* = \frac{-\lambda_2}{2r}$ from the HJB.

$\lambda_2 = s_2x + s_3\dot{x}$. So

$$u^* = \frac{-s_2x}{2r} - \frac{s_3\dot{x}}{2r}$$

This is the linear state feedback. Its the optimal control policy in terms of s_2 and s_3 , with gains $\frac{-s_2(t)}{2r}$ on position and $\frac{-s_3(t)}{2r}$ on velocity.

2. (10 points) Pick a system from a paper that you have found on a topic that you are interested in, potentially the topic you will choose for your final project.

(a) What is/are the paper(s) you are drawing from?

Ans: This paper presents a dynamic model of a quadcopter UAV with friction effects, and a two level input-output control strategy combining linearization and PID controllers

(b) What are the dynamics of the system in question? What are the inputs and outputs?

Ans:

- **Inputs:**

1. Total thrust F
2. Torques $[\rho_\phi, \rho_\theta, \rho_\psi]$

- **Outputs:**

- Position $\xi = [x, y, z]^T$
- Orientation $\eta = [\phi, \theta, \psi]^T$

The quadcopter is modelled as a nonlinear underactuated system. The euler-lagrange equations are used to derive the equations of motion, and they also include frictionless terms.

By friction terms, they mean simplified models for including air resistance and aerodynamic drag

(c) What is the system trying to achieve?

Ans: The paper tries to solve the following

1. Hovering, take-off and landing
2. Follow a predetermined path in 3d, or geometric trajectories
3. Stabilizing attitude, namely the pitch, roll and yaw.

(d) What are the challenges for the system?

Ans: The problems in the paper are

- **Problem 1:** Coupling between rotational and translational motions, meaning that in order to move forward, the nose must first tip down, as the only thrusters points down.
- **Problem 2:** Nonlinear dynamics. This means that there is not a function that can connect the inputs to the outputs, like a traditional function $f(x) = kx^n$
- **Problem 3:** Avoiding local linearization, i.e. when changing an action, from hovering to travelling, the local linearization becomes invalid. This is the same logic as using Taylor expansions of a function

around a point. The approximation works great locally, but move away and the error between approximation and true f grows large.

- **Problem 4:** Friction and air drag. Also in a realistic situation, you need to figure out how to account for essentially random gusts of wind that puts the drone out of balance.
- **Problem 5:** Need for fast and stable inner-loop control to support outer loop control
- **Problem 6:** Singularities when $\phi, \theta = \pm \frac{\pi}{2}$

- (e) What control tools can help the system overcome the challenges in achieving its goals?

Ans: For choosing the right gains, they propose that the gains follow a characteristic equation. They do this by pole placement, and finding the gains K that solves the characteristic equation.

For the singularities for the angles ϕ and θ , they define angle limiters, which is simple enough. They could also use quaternions, for more continuous movement, but avoided it for simplicity.

To solve the stability problem, they make an inner- and outer loop. The inner loop controls the quadcopters orientation and attitude, while the outer control deals with the position.

For the input-output linearisation, the paper proposes to transform the non-linear dynamics into linear decoupled systems. Unlike Taylor series, which only works locally, this method is globally valid, as each degree of freedom to be tuned is not a function of another variable.

3. (20 points) Suppose that $f : \mathbb{R} \mapsto \mathbb{R}$ is convex, and $a < b$ are in the domain of f .

- (a) Show that

$$f(x) \leq \frac{b-x}{b-a}f(a) + \frac{x-a}{b-a}f(b) \quad (4)$$

for all $x \in [a, b]$

Ans: A criteria for a function to be convex is that

$$f(x) \leq \lambda f(a) + (1 - \lambda) f(b)$$

$\forall x \in \mathbb{R}$

If we set $x = \lambda a + (1 - \lambda) b$ where $\lambda = \frac{b-x}{b-a}$, then

$$\begin{aligned} 1 - \lambda &= 1 - \frac{b-x}{b-a} \\ &= \frac{(b-a) - (b-x)}{b-a} \\ &= \frac{x-a}{b-a} \end{aligned}$$

And we get that

$$f(x) = f(\lambda a + (1 - \lambda) b) \leq \lambda f(a) + (1 - \lambda) f(b)$$

Which is the criteria for a function to be convex, and it is proved.

(b) Show that

$$\frac{f(x) - f(a)}{x - a} \leq \frac{f(b) - f(a)}{b - a} \leq \frac{f(b) - f(x)}{b - x} \quad (5)$$

for all $x \in (a, b)$. Draw a sketch that illustrates this inequality

Ans: If we pick up our old calculus text book, we find that a function $f : \mathbb{R} \rightarrow \mathbb{R}$ iff. $a \leq b \forall c \in \mathbb{R}$

$$\frac{f(c) - f(a)}{c - a} \leq \frac{f(b) - f(c)}{b - c}$$

If we assume that f is convex, then the line between the points $(a, f(a))$ and $(b, f(b))$ will never be under the point $(c, f(c))$.

1. Lets now denote the slope of the line $d((a, f(a)), (b, f(b)))$ as k . In this case, we use the $\frac{\text{rise}}{\text{run}}$ formula to get that $\frac{f(b)-f(a)}{b-a} = k$
2. The line $d((c, f(c)), (a, f(a)))$ as the line $\frac{f(c)-f(a)}{c-a} = k_1$
3. And finally, the line $d((b, f(b)), (c, f(c)))$ as the line $\frac{f(b)-f(c)}{b-c} = k_2$

If f is truly convex, then $k_1 \leq k \leq k_2$. Therefore,

$$\frac{f(c) - f(a)}{c - a} \leq \frac{f(b) - f(a)}{b - a} \leq \frac{f(b) - f(c)}{b - c}$$

Now we can just replace c with x , because this inequality is valid $\forall x \in (a, b)$

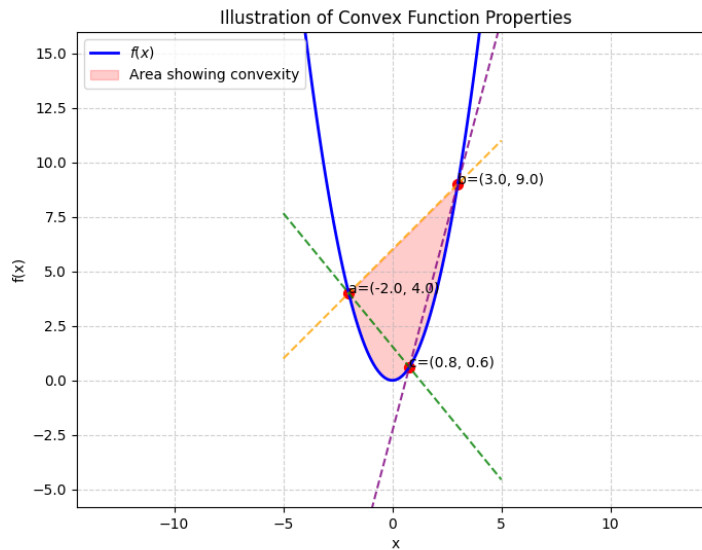


Fig. 1: Building model schematic

(c) Suppose that f is differentiable. Use the result in (b) to show that

$$f'(a) \leq \frac{f(b) - f(a)}{b - a} \leq f'(b). \quad (6)$$

Ans: From b we have that

$$\frac{f(c) - f(a)}{c - a} \leq \frac{f(b) - f(a)}{b - a} \leq \frac{f(b) - f(c)}{b - c}$$

We know from a definition of derivatives in our calculus book, that

$$f'(a) = \lim_{x \rightarrow a} \frac{f(x) - f(a)}{x - a}$$

. Then since $x - b \leq 0$ we have that

$$f'(b) = \lim_{x \rightarrow b} \frac{f(b) - f(x)}{b - x}$$

From this we get what we wanted to prove, if we allow x to approach both a and b respectively

(d) Suppose f is twice differentiable. Use the result in (c) to show that $f''(a) \geq 0$ and $f''(b) \geq 0$.

Ans: From c), we got that $f'(a) \leq \frac{f(b)-f(a)}{b-a} \leq f'(b)$. Now the second derivative becomes

$$f''(a) = \lim_{h \rightarrow 0} \frac{f'(a+h) - f'(a)}{h}$$

Now applying the mean value theorem to the interval $[a, a+h]$, we get $\exists$ some $c \in (a, a+h)$ s.t.

$$f'(c) = \frac{f(a+h) - f(a)}{h}$$

But from our inequality with $x = a+h$, we get that $f'(a) \leq f'(c)$. Since $c > a$ and $f'(c) \geq f'(a)$, the rate of change is non decreasing at a . Therefore,

$$f'(a+h) \geq f'(a)$$

which implies that $\frac{f'(a+h)-f'(a)}{h} \geq 0$ which is what we wanted to show. A similar argument is made for $f''(b)$

4. (20 points) (LQR for Buildings) Consider a multi-zone building as in Fig. 2.

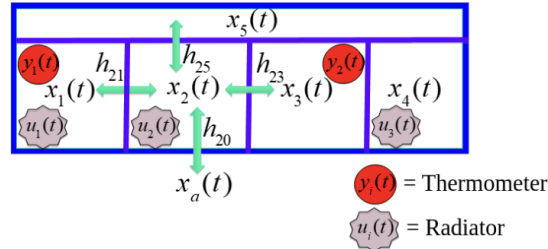


Fig. 2: Building model schematic

- (a) Write down the dynamical equations of the building in state-space form, including a term for disturbances $d(t)$, and for the ambient temperature $x_a(t)$, assuming the parameters are

$$\frac{h_{ij}}{C_i} = \begin{cases} 10 \text{ W/}^\circ\text{K} & i \text{ is adjacent to } j \\ 0 & \text{else} \end{cases} \quad (7)$$

$$C_i = 1.08 \times 10^6 \text{ J/}^\circ\text{K} \quad (8)$$

- (b) Consider an LQR cost function of the form

$$\int_0^T [(x - \bar{x})^T Q (x - \bar{x}) + u^T R u] dt, \quad (9)$$

where $T = 24\text{h} \times 60\frac{\text{m}}{\text{h}} \times 60\frac{\text{s}}{\text{m}} = 86400\text{s}$ is one day in seconds. Set $\bar{x} = 20^\circ$, and write the dynamics in the coordinates $z := x - \bar{x}$.

- (c) Note that driving the variable $z(t) \rightarrow 0$ is therefore equivalent to driving $x(t) \rightarrow 20^\circ$. Write a `matlab` script to solve the finite-time horizon LQR problem. Select a positive semi-definite Q and a positive-definite R that gives you “decent” results. Assume the initial conditions, disturbance, and ambient temperature follow:

$$x_i(0) = 15 - i \times \text{sign}\{(-1)^i\} \quad [^\circ \text{C}] \quad (10)$$

$$d(t) = \max \left[0, 500 \sin \left(\frac{2\pi t}{86400\text{s}} \right) - 300 \right] \quad [\text{W}] \quad (11)$$

$$x_a(t) = 20^\circ + 5 \sin \left(\frac{2\pi t}{86400\text{s}} \right) \quad [^\circ \text{C}] \quad (12)$$

where the sign function is:

$$\text{sign}(x) = \begin{cases} 1 & x > 0 \\ -1 & x < 0 \\ 0 & x = 0. \end{cases} \quad (13)$$

Note: If solving the finite-time problem proves too difficult, feel free to solve the infinite-time problem instead. `Matlab` has a built-in function called `lqr` that produces the (constant) gain K ; read the documentation for details.

- (d) Provide:

- A description of your simulation code
- Plots of $d(t)$, $x_a(t)$, $x_i(t)$, and $u(t)$.
- A discussion of the effects of varying Q and R

Ans:

- (a) To find the dynamical equations of the building in state-space form, we have to find the system on the form

$$\dot{x} = Ax + Bu, y = Cx$$

The individual room equations are

$$C_i \dot{x}_i(t) = \sum_{j \in \text{neighbors}} h_{ij} (x_j(t) - x_i(t)) + h_{i0} (x_a(t) - x_i(t)) + u_i(t) + d_i(t)$$

We rearrange this to get

$$\dot{x} = \sum_j \frac{h_{ij}}{C_i} (x_j(t) - x_i(t)) + \frac{h_{i0}}{C_i} (x_a(t) - x_i(t)) + \frac{u_i(t)}{C_i} + \frac{d_i(t)}{C_i}$$

We have 5 rooms, thus 5 states, so $x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \\ x_5(t) \end{bmatrix}$, 3 controllers that

regulate temperature, $u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \\ u_3(t) \end{bmatrix}$ and finally, one disturbance term for

every room $d(t) = \begin{bmatrix} d_1(t) \\ d_2(t) \\ d_3(t) \\ d_4(t) \\ d_5(t) \end{bmatrix}$

We incorporate the disturbance term, ambient temperatures and the controls

(radiators) into a single vector $u^* = \begin{bmatrix} u \\ d \\ x_a \end{bmatrix}$

$$\frac{h_{ij}}{C_i} = \begin{cases} 10W/^{\circ}K, & \text{if } i \text{ adjacent to } j \\ 0, & \text{else} \end{cases}$$

This makes A to be a banded matrix. Since all rooms are adjacent to room 5, the matrix A becomes

$$A = \begin{bmatrix} -\sum_j \frac{h_{1j}}{C_1} & \frac{h_{12}}{C_1} & 0 & 0 & \frac{h_{15}}{C_1} \\ \frac{h_{21}}{C_2} & -\sum_j \frac{h_{2j}}{C_2} & \frac{h_{23}}{C_2} & 0 & \frac{h_{25}}{C_2} \\ 0 & \frac{h_{32}}{C_3} & -\sum_j \frac{h_{3j}}{C_3} & \frac{h_{34}}{C_3} & \frac{h_{35}}{C_3} \\ 0 & 0 & \frac{h_{43}}{C_4} & -\sum_j \frac{h_{4j}}{C_4} & \frac{h_{45}}{C_4} \\ \frac{h_{51}}{C_5} & \frac{h_{52}}{C_5} & \frac{h_{53}}{C_5} & \frac{h_{54}}{C_5} & -\sum_j \frac{h_{5j}}{C_5} \end{bmatrix}$$

$$= \begin{bmatrix} -30 & 10 & 0 & 0 & 10 \\ 10 & -40 & 10 & 0 & 10 \\ 0 & 10 & -40 & 10 & 10 \\ 0 & 0 & 10 & -30 & 10 \\ 10 & 10 & 10 & 10 & -50 \end{bmatrix}$$

Since there are three actuators in rooms 1, 2 and 4, the B matrix is written as

$$B = \begin{bmatrix} \frac{1}{C_1} & 0 & 0 \\ 0 & \frac{1}{C_2} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{C_4} \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{1.08 \times 10^6} & 0 & 0 \\ 0 & \frac{1}{1.08 \times 10^6} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{1.08 \times 10^6} \\ 0 & 0 & 0 \end{bmatrix}$$

We have the vector b that takes into account the ambient coupling of temperatures.

$$b = \begin{bmatrix} H_{10} \\ H_{20} \\ H_{30} \\ H_{40} \\ H_{50} \end{bmatrix} = \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix}$$

and D is a diagonal matrix with $\frac{1}{C_i} = \frac{1}{1.08 \times 10^6}$ for disturbance inputs.

If we now follow the notes on building control from the lecture slides of optimal control (lecture 4), we obtain the dynamic equations $\dot{x} = Ax(t) + Bu(t) + bx_a(t) + Dd(t)$

- (b) For this problem, we note that $z = x - \bar{x}$, then $x = z + \bar{x}$ and $\dot{x} = \dot{z}$ since $\bar{x}$ is a constant temperature of $20^\circ C \approx 293^\circ K$ since $1^\circ C = 1K$ (K = Kelvin) so we get the dynamics in the z coordinates as

$$\begin{aligned} \dot{z} &= A(z + \bar{x}) + Bu \\ &= Az + A\bar{x} + Bu \end{aligned}$$

$A\bar{x}$ can be seen as the forcing term towards the reference temperature in the system.

- (c) Solving the finite-time horizon problem seems to me very hard, as we don't have any initial points to work backwards from. So I just used the infinite-time horizon

I will attach the matlab script below. From experimenting, I see that a larger value of Q on the diagonals, makes the system more stable, and temperatures in all rooms converge to 20 degrees. Moderate R values allow for moderate control, without spending too much energy. For large values on the diagonal

of Q , we prioritize error tracking accuracy. This leads to smaller steady state errors. Values Can go as far as 1000

For the R matrix, we should have smaller values, as far down as 100. What this leads to, is sufficient control action. It means that we dont overdo control, and dont underdo, such that we dont oscillate the control effort.

We see that the Q and R values become weights that account for how important we want to optimize different parts of our system

- (d) The plant system from the exercise, $\dot{x} = Ax + Bu$, $y = Cx$. From b), we got that $\dot{z} = Az + A\bar{x} + Bu$. Because of the disturbance terms, we have several terms to include

$$B_d = \begin{bmatrix} 1.08 \times 10^{-6} & 0 & 0 & 0 & 0 \\ 0 & 1.08 \times 10^{-6} & 0 & 0 & 0 \\ 0 & 0 & 1.08 \times 10^{-6} & 0 & 0 \\ 0 & 0 & 0 & 1.08 \times 10^{-6} & 0 \\ 0 & 0 & 0 & 0 & 1.08 \times 10^{-6} \end{bmatrix}$$

$$\text{and } B_u = \begin{bmatrix} 1.08 \times 10^{-6} & 0 & 0 \\ 0 & 1.08 \times 10^{-6} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1.08 \times 10^{-6} \\ 0 & 0 & 0 \end{bmatrix} \text{ and finally } B_a = \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix}$$

We are. then allowed to rewrite our system as

$$\dot{z} = Az + A\bar{x} + B_d d + B_a x_a$$

$$\dot{z} = Az + B_u u^* + B_d d + \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix} \left(293 + 5 \sin \left(\frac{2\pi t}{86400s} \right) \right) - 293 * \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix}$$

$$\dot{z} = Az + B_u u^* + B_d d + \begin{bmatrix} 10 \\ 10 \\ 10 \\ 10 \\ 10 \end{bmatrix} * 5 \sin \left(\frac{2\pi t}{86400s} \right)$$

The formula for the initial conditional temperature in all 5 rooms is provided, and in degrees they are (17, 13, 18, 11, 20) and in z coordinates, they are (-4, -7, -2, -9, 0)

For the LQR system design, I use the lqr function with $Q = I * q$ and $R = I * r$. The difference ($q - r$) indicates if I favor tracking errors or

control effort of the radiators more. In my case, with $q = 1e6$ and $r = 1e2$, the balance is $\frac{q}{r} = 1000$.

For observer design, we must check that the system is observable. The LQR controller $u = -Kz$ requires full state knowledge. With only two temperature measurements in two rooms, we need full state observability over all the states $(z)_i^{1to5}$. We have the true system $\dot{z} = Az + Bu + \text{disturbance term}$, and in parallel, we have an observer running $\dot{\hat{z}} = A\hat{z} + Bu + L(y - C\hat{z})$

To derive the augmented matrix, we have the true state dynamic $\dot{z} = Ax + Bu + B_{ext}w$ where $w = \begin{bmatrix} d \\ x_a \end{bmatrix}$. The observer dynamics is defined as

$$\begin{aligned} \frac{d\hat{z}}{dt} &= A\hat{z} + Bu + L(Cz - C\hat{z}) \\ &= \hat{z}(A - LC) + Bu + LCz \end{aligned}$$

The control using the estimates is

$$u = -K\hat{z}$$

Substituting this to the true dynamics gives

$$\dot{z} = Ax + B(-K\hat{z}) + B_{ext}w$$

Also substituting into the observer,

$$\frac{d\hat{z}}{dt} = LCz + (A - LC)\hat{z} - BK\hat{z}$$

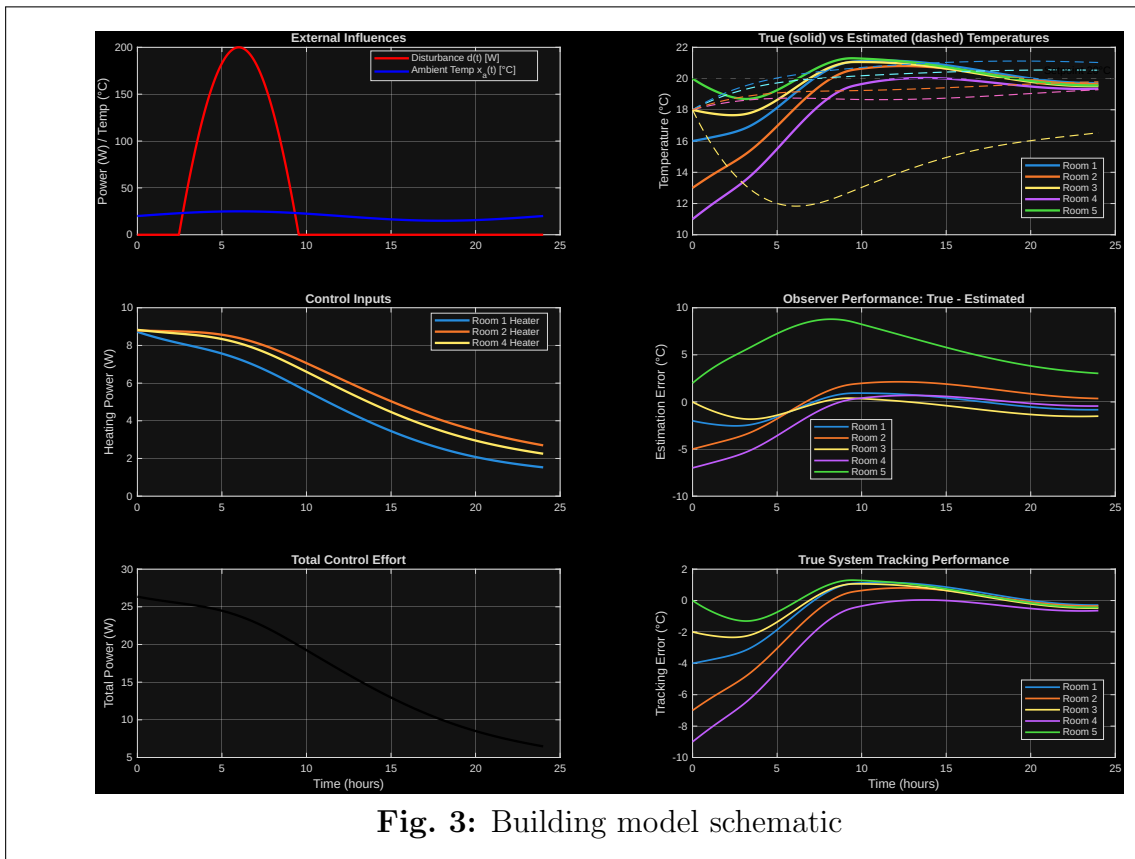
And now pulling it into matrix form we get

$$A_{aug} = \begin{bmatrix} A & -B_uK \\ LC & A - B_uK - LC \end{bmatrix}$$

Now

- A_{aug} is a 10×10 matrix
- B_{aug} is a 10×6 matrix
- z_{aug} is a 10×1 matrix

Now the plot looks like this, and it should include everything demanded



5. (10 points) Suppose that $Q = \mathbf{0}$, the zero matrix, and $R = I$. Prove that

$$x_0^T S_0 x_0 = \min_{\{u_k\}_{k=0}^{T-1}} \sum_{k=0}^{T-1} \|u_k\|_2^2, \quad (14)$$

meaning that $J_0 = x_0^T S_0 x_0$ is the minimum control energy required to drive the system to zero.

Ans: We have the discrete LQR with dynamic programming, and an associated cost index

$$J_i = x_N^T S_N x_N + \sum_{k=i}^{N-1} (x_k^T Q x_k + u_k^T R u_k)$$

we are given that $Q = 0$ and $R = I$. Then we get the cost index

$$J_i = x_N^T S_N x_N + \sum_{k=i}^{N-1} u_k^T u_k$$

We want to find the control sequence u^* that minimizes the cost function J_i . So our problem is the optimization problem

$$J_0^* = \min_u J_0(u)$$

s.t. $x_{k+1} = Ax_k + Bu_k$. This is largely already proved on page 270-271 in Optimal Control by Lewis, but I will show the main results. We will have big help of the principle of optimality, which states that along the optimal path, any subpath along that trajectory, is also an optimal trajectory.

So we have that $J_N^* = x_N^T S_N x_N$. Now decrement to N-1 and get $J_{N-1} = u_{N-1}^T u_{N-1} + x_N^T S_N x_N$ and substitute in $x_{k+1} = Ax_k + Bu_k$.

$$J_{N-1} = u_{N-1}^T u_{N-1} + (Ax_{N-1} + Bu_{N-1})^T S_N (Ax_{N-1} + Bu_{N-1})$$

Now we optimize, and since the cost function is convex, we know we find the minimum when $\frac{\partial J_{N-1}}{\partial u} = 0 = u_{N-1} + B^T S_N (Ax_{N-1} + Bu_{N-1})$ and get

$$\begin{aligned} 0 &= u_{N-1} + B^T S_N Ax_{N-1} + B^T S_N Bu_{N-1} \\ &= u_{N-1} (I + B^T S_N B) + B^T S_N Ax_{N-1} \\ u_{N-1}^* &= -(I + B^T S_N B)^{-1} + B^T S_N Ax_{N-1} \end{aligned}$$

If we now define the gain as

$$K = -(I + B^T S_N B)^{-1} + B^T S_N A$$

we get that

$$u_{N-1}^* := K_{N-1} x_{N-1}$$

which is the optimal cost to go from step number $N - 1$. We can go recursively back and find that

$$\begin{aligned}
 J_0^* &= x_0^T S_0 x_0 \\
 &= x_N^T S_N x_N + \sum_{k=0}^{N-1} (u_k^*)^T u_k^* \\
 &= \sum_{k=0}^{N-1} \|u_k^*\|_2^2
 \end{aligned}$$

Now we just also use the fact that $J_0^* = \min_u J_0(u)$ and get that

$$x_0^T S_0 x_0 = \min_u \sum_{k=0}^{N-1} \|u_k\|_2^2$$

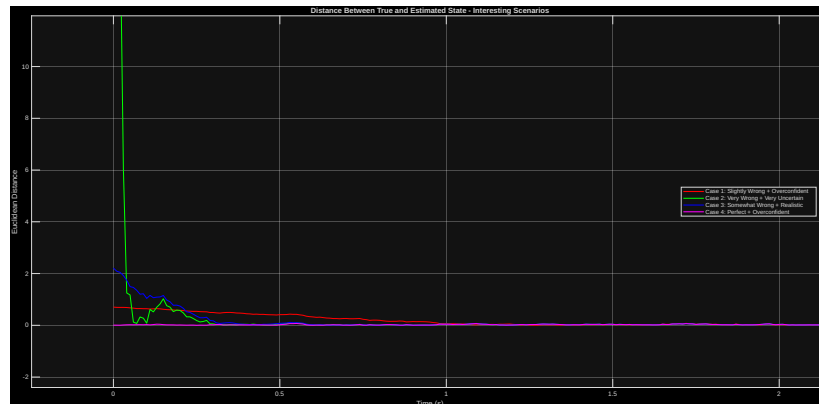
6. (10 points) *Crassidis & Junkins* Problem 3.14

Ans: In this problem, we can imagine we have sensors, and an idea of where we are. The problem is to analyze the difference between what state we think we are in, and the true state. We use two different parameters to analyze the different cases. The initial conditions and the covariance matrix. The initial condition means where we think we are, and the covariance matrix means how sure we are of that. A covariance matrix with 0 as elements, means we are absolutely sure we are in the proper initial condition. So if the distance between the true state and the measured state is

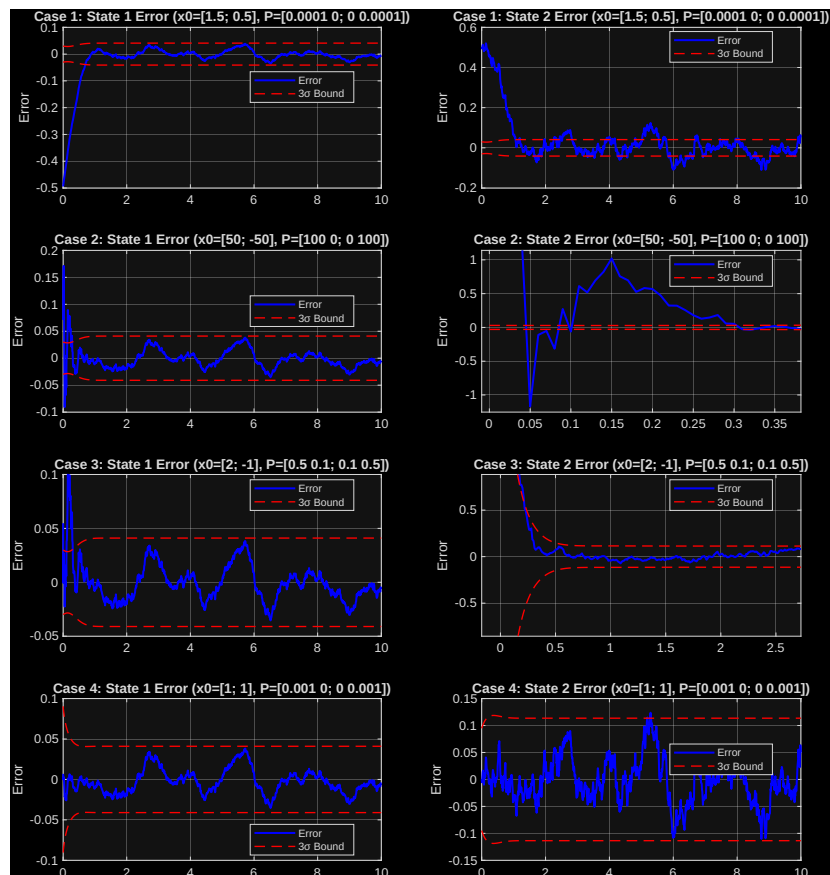
$$\|x_{true} - x_{hat}\| = c$$

Then the speed that c converges to 0, will be determined by the covariance matrix itself.

Here is the plot that shows the distance in norms for several initial conditions and covariance matrices.



And here is the plot that shows the 3sigma bounds



The code is attached in the appendix.

7. (10 points) *Crassidis & Junkins* Problem 3.15

Ans: What we can see from the new H matrix, is that convergence for the (l^2) norms between the states true and predicted states almost instantly, in a matter of milliseconds go to 0. The reason is because of observability.

1. *Case 1*

- For state 1, we see that its taking more time to converge to between the 3σ bounds
- For state 2, we have the same story, longer time, with higher variance, but it seems to be more within the 3σ bounds

2. *Case 2*

- For state 1, we have better performance, with tighter 3σ bounds, and also faster convergence.
- For state 2, its performing alot better, with the previous measurement model having really bad convergence. But there is still a lot of outliers in the data, because its not largely contained within the 3σ bounds

3. *Case 3*

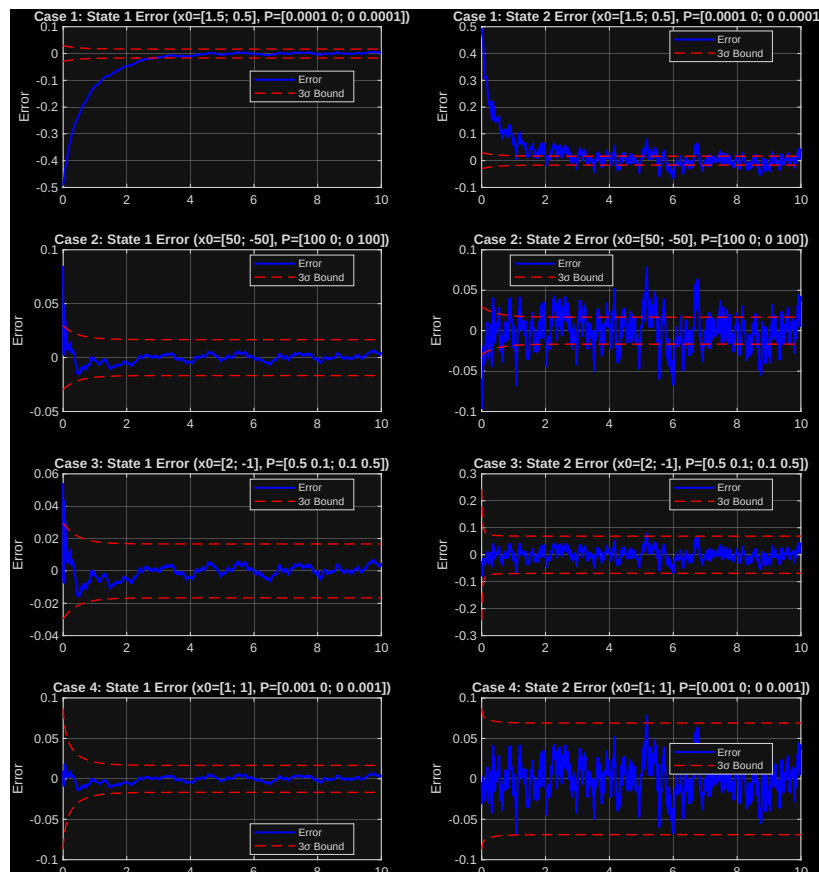
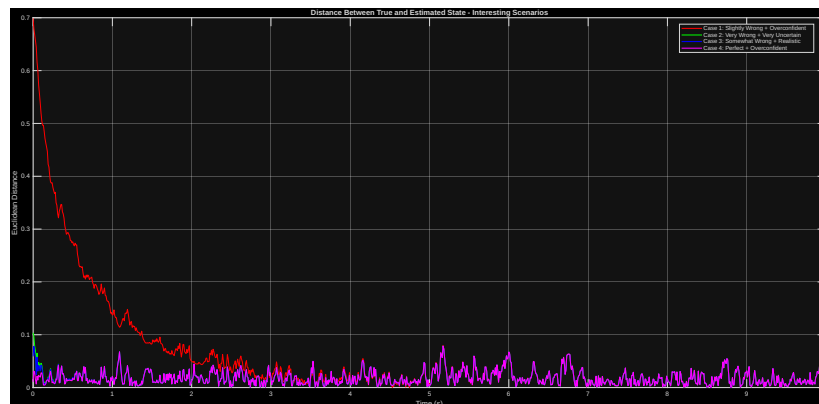
- For state 1, we have better performance for convergence and variance. Great success
- For state 2, same story, faster convergence, and smaller 3σ bounds.

4. *Case 4*

- For state 1, we have better convergence and tighter bounds
- For state 2, we have also better convergence, and smaller variance with slightly tightet bounds

So overall, we can say confidently that the model has largely improved, by providing a measurement model that gives better observability.

Here is the figures from the matlab script



8. (10 points) *Crassidis & Junkins* Problem 7.4

Ans:

Quick note, Im just ramming up the important math at the beginning. You can skip to where I say 'Furhtermore' for direct results from simulation. Instead of using euler angles, a nice representation of attitude is using quaternions.

$$\vec{q} = [q_1, q_2, q_3, q_4] = [\epsilon, q_4]$$

where ϵ is the vector part. The kinematics of the quaternion follows

$$\frac{dq}{dt} = \frac{1}{2}\Omega(\vec{\omega})q$$

where ω is the angular velocity. From mechanics, the velocity of a particle rotating about an axis, is derived by $\vec{v} = \vec{\omega} \times \vec{r}$. Not important here. Just meant to illustrate that its a measure of speed, given by radians per unit of time.

For the gyro measurements, we used a first order markov process

$$\vec{\omega}_{measured} = \omega_{true} + \beta + \eta_v \quad (\text{the measurement noise})$$

$$\frac{d\beta}{dt} = \eta_u \quad (\text{bias drift noise})$$

where beta is the gyro bias vector.

For the error state formulation, we use the multiplicative error to maintain the quaternion normalization constraint, $\delta\vec{q} = \vec{q} \otimes \hat{\vec{q}}^{-1}$

The error state dynamics are given as linearized error dynamics,

- $\frac{d\delta\alpha}{dt} = -[\omega \times] \delta\alpha - \Delta\beta - \eta_v$
- $\frac{d\Delta\beta}{dt} = \eta_u$

where $[\omega \times]$ is the skew symmetric matrix given by
$$\begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}$$

And now for the extended kalman filter implementation, we start by defining the state vector $\vec{x} = [\delta\alpha, \Delta\beta^T]$. For each step in our discretized timeline, we propagate the new quaternion as

$$\hat{q}_{k+1}^- = \exp\left(\frac{1}{2}\Omega(\hat{\omega}_k^+) \Delta t\right) \hat{q}_k^+$$

where the exponential is computed as

$$\exp\left(\frac{1}{2}\Omega(\omega)\delta t\right) = \begin{bmatrix} \cos(1/2\|\omega\|\Delta t) I - [\phi \times] & \psi \\ -\psi^T & \cos(1/2\|\omega\|\Delta t) \end{bmatrix}$$

with $\psi = \sin(1/2\|\omega\|\Delta t) * \frac{\omega}{\|\omega\|}$

The process noise covariation (eq. 7.46) is given by

$$Q = \begin{bmatrix} (\sigma_v^2 \Delta t + \frac{1}{2} \sigma_u^2 \Delta t^3) & (\frac{1}{2} \sigma_u^2 \Delta t^3) * I \\ (\frac{1}{2} \sigma_u^2 \Delta t^2) I & (\sigma_u^2 \Delta t) I \end{bmatrix}$$

And is in this case a 6×6 shaped matrix

The sensor measurement updates are given by

$$b_i = A(q)r_i + v_i$$

for $i = 1, 2, 3$. $A(q)$ is the attitude matrix and r_i are the reference vectors. The measurement matrix is given by

$$H = \begin{bmatrix} [A(\hat{q})r_1 \times] & 0 \\ [A(\hat{q})r_2 \times] & 0 \\ [A(\hat{q})r_3 \times] & 0 \end{bmatrix}$$

where again $[v \times]$ is the skew symmetric matrix.

Furthermore, for the implementation strategy, we assume a constant angular velocity, and measurements are made with gyros that contain a bias. The angular velocity is given by $\omega_{true} = [0.001, 0.002, 0.0005]$ rad/s. This creates a slow and accumulating rotation of all the axes, which is an attempt of more realistic dynamics. This is implemented with the function `exact attitude`. I also assume constant gyro biases $\beta_{true} = [0.1, 0.1, 0.1]$ deg/hr = $[4.848 * 10^{-7}, 4.848 * 10^{-7}, 4.848 * 10^{-7}]$ rad/s

The sampling rate for the gyroscope is 1 Hz, which makes $dt = 1$ second.

Now we can start analyzing the filter performance

0.1 baseline scenario

This scenario represents balanced and realistic sensor performance.

- $\sigma_u = 1 * 10^{-5}$ rad/s/ $\sqrt{\text{Hz}}$
- $\sigma_v = 1 * 10^{-4}$ rad/s/ $\sqrt{\text{Hz}}$

0.2 high σ_v

This scenario tests sensitivity to gyro measurement noise

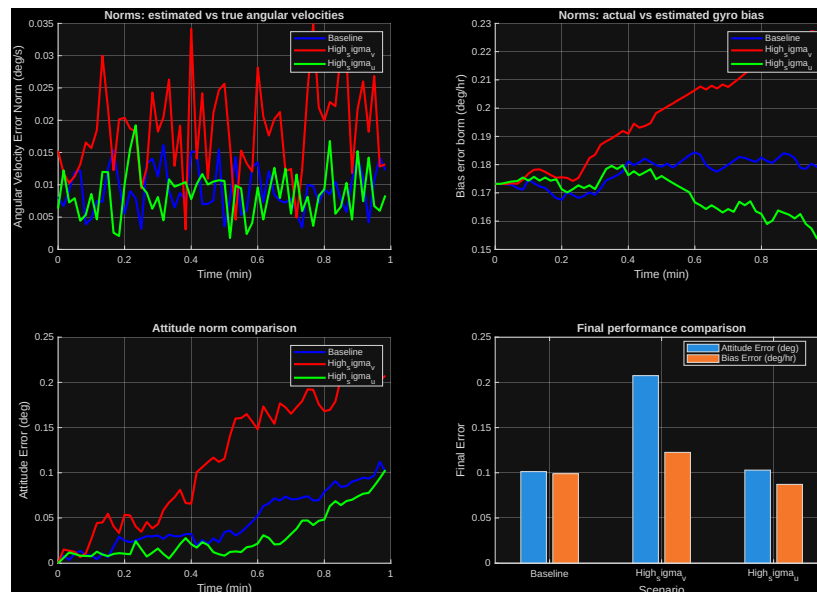
- $\sigma_u = 1 * 10^{-5} \text{ rad/s}/\sqrt{Hz}$
- $\sigma_v = 2 * 10^{-4} \text{ rad/s}/\sqrt{Hz}$

0.3 high σ_u

This scenario tests sensitivity to bias instability

- $\sigma_u = 2 * 10^{-5} \text{ rad/s}/\sqrt{Hz}$
- $\sigma_v = 1 * 10^{-4} \text{ rad/s}/\sqrt{Hz}$

The following plot ($dt = 1$)



suggests that the filter is more sensitive to increases in measurement noise, than process noise. σ_u was related to the gyro bias drift, and σ_v to the gyro measurement noise. The difference in angular velocities estimates seem to be really stable. If I decrease dt (thus increasing discretization rate) the plot of norms of angular velocities becomes unclear. For the attitude error and gyro bias, the errors seem to diverge more, with the high measurement noise scenario diverging

in both cases. So the readings from our sensor, contain a significant amount of random unpredictable error. It does not seem that any increase in process noise has any significant impact on performance, as the norm stays relatively low, at least compared to measurement noise. This is also illustrated by the final comparison plot

1 appendix

1.1 Python code for Exc 3b

```

1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  x = np.linspace(-5, 5, 100)
5
6  def plot_convex(a, b, f):
7      if b <= a:
8          raise ValueError("b must be greater than a")
9
10     y = f(x)
11     c = np.random.choice(np.linspace(a, b, 10))
12
13     k = (f(b) - f(a)) / (b - a)
14     k_1 = (f(c) - f(a)) / (c - a)
15     k_2 = (f(b) - f(c)) / (b - c)
16
17     secant = f(a) + k * (x - a)
18     secant_left = f(a) + k_1 * (x - a)
19     secant_right = f(c) + k_2 * (x - c)
20
21     plt.figure(figsize=(8, 6))
22     plt.plot(x, y, label=r"$f(x)$", color="blue", linewidth=2)
23     plt.plot(x, secant, "--", color="orange", alpha=0.8)
24     plt.plot(x, secant_left, "--", color="green", alpha=0.8)
25     plt.plot(x, secant_right, "--", color="purple", alpha=0.8)
26
27     plt.scatter([a, b, c], [f(a), f(b), f(c)], color="red", s=50)
28     plt.annotate(f"a=({a:.1f}), {f(a):.1f}", (a, f(a)))
29     plt.annotate(f"b=({b:.1f}), {f(b):.1f}", (b, f(b)))
30     plt.annotate(f"c=({c:.1f}), {f(c):.1f}", (c, f(c)))
31

```

```

32     x_fill = np.linspace(a, b, 50)
33     y_fill = f(x_fill)
34     secant_fill = f(a) + k * (x_fill - a)
35     plt.fill_between(x_fill, y_fill, secant_fill, alpha=0.2,
36                     color='red', label='Area showing convexity')
37
38     plt.title("Illustration of Convex Function Properties")
39     plt.xlabel("x")
40     plt.ylabel("f(x)")
41     plt.legend()
42     plt.grid(True, linestyle='--', alpha=0.6)
43     plt.axis('equal')
44     plt.show()
45
46 plot_convex(-2, 3, lambda x: x**2)

```

1.2 Matlab code for exercise 4

```

1     nRooms = 5;
2     nControls = 3;
3     nMeasurements = 2;
4
5     q = 1e4;
6     r = 1e2;
7     x_bar = 20 * ones(nRooms, 1);
8     T = 86400;
9     t = linspace(0, T, 1000);
10    c = 1/(1.08e6);
11
12    A = [-30 10 0 0 10;
13         10 -40 10 0 10;
14         0 10 -40 10 10;
15         0 0 10 -30 10;
16         10 10 10 10 -50] * c;
17
18    B = [1 0 0;
19         0 1 0;
20         0 0 0;
21         0 0 1;
22         0 0 0] * c;
23
24    C = [0 1 0 0 0;
25         0 0 0 1 0];

```



```

26
27 Q = q * eye(nRooms);
28 R = r * eye(nControls);
29 [K, P, E] = lqr(A, B, Q, R);
30
31 if rank(observ(A, C)) == nRooms
32     fprintf('System is observable.\n');
33 else
34     fprintf('System is NOT observable\n');
35 end
36
37 obs_poles_multiplier = 2;
38 desired_obs_poles = obs_poles_multiplier * max(real(E)) * linspace(1, 1.5,
39 L = place(A', C', desired_obs_poles)');
40
41 B_ambient = [10; 10; 10; 10; 10] * c;
42 D_disturbance = eye(5) * c;
43 x_a = @(t) 20 + 5*sin(2*pi*t/86400);
44 d_func = @(t) max(0, 500 * sin(2 * pi*t/86400) - 300);
45
46 ssign = @(x) (x>0) - (x<0);
47 x0_true = zeros(nRooms, 1);
48 for i = 1:nRooms
49     x0_true(i) = 15 - i * ssign((-1)^i);
50 end
51 z0_true = x0_true - x_bar;
52
53 x0_est = 18 * ones(nRooms, 1);
54 z0_est = x0_est - x_bar;
55
56 z0_aug = [z0_true; z0_est];
57
58 augmentedDynamics = @(t, z_aug) [
59     A * z_aug(1:5) + B * (-K * z_aug(6:10)) + ...
60     B_ambient * (x_a(t) - 20) + ...
61     D_disturbance * d_func(t) * ones(5,1);
62
63     A * z_aug(6:10) + B * (-K * z_aug(6:10)) + ...
64     L * (C * z_aug(1:5) - C * z_aug(6:10))
65 ];
66
67 [t_sim, z_aug] = ode45(@(t,z) augmentedDynamics(t, z), t, z0_aug);
68
69 z_true = z_aug(:, 1:5)';
70 z_est = z_aug(:, 6:10)';

```

```

71
72 x_true = z_true + x_bar;
73 x_est = z_est + x_bar;
74
75 u_sim = zeros(length(t_sim), nControls);
76 for i = 1:length(t_sim)
77     u_sim(i,:) = (-K * z_est(:,i))';
78 end
79
80
81 t_hours = t_sim / 3600;
82
83 figure('Position', [100, 100, 1400, 1000]);
84
85 subplot(3, 2, 1);
86 d_values = arrayfun(d_func, t_sim);
87 x_a_values = arrayfun(x_a, t_sim);
88 plot(t_hours, d_values, 'r-', 'LineWidth', 2); hold on;
89 plot(t_hours, x_a_values, 'b-', 'LineWidth', 2);
90 ylabel('Power(W)/Temp( C )');
91 title('External Influences');
92 legend('Disturbance_d(t)[W]', 'Ambient_Temp_x_a(t)[ C ]', 'Location', 'best');
93 grid on;
94
95 subplot(3, 2, 2);
96 plot(t_hours, x_true, 'LineWidth', 2); hold on;
97 plot(t_hours, x_est, '--', 'LineWidth', 1);
98 yline(20, '--k', 'Setpoint_20 C ', 'LineWidth', 1);
99 ylabel('Temperature( C )');
100 title('True(solid) vs Estimated(dashed) Temperatures');
101 legend('Room_1', 'Room_2', 'Room_3', 'Room_4', 'Room_5', 'Location', 'best');
102 grid on;
103
104 subplot(3, 2, 3);
105 plot(t_hours, u_sim, 'LineWidth', 2);
106 ylabel('Heating_Power(W)');
107 title('Control Inputs');
108 legend('Room_1_Heater', 'Room_2_Heater', 'Room_4_Heater', 'Location', 'best');
109 grid on;
110
111 subplot(3, 2, 4);
112 est_errors = x_true - x_est;
113 plot(t_hours, est_errors, 'LineWidth', 1.5);
114 ylabel('Estimation_Error( C )');
115 title('Observer Performance: True - Estimated');

```

```

116 legend('Room_1', 'Room_2', 'Room_3', 'Room_4', 'Room_5', 'Location', 'best
117 grid on;
118
119 subplot(3, 2, 5);
120 total_power = sum(u_sim, 2);
121 plot(t_hours, total_power, 'k-', 'LineWidth', 2);
122 ylabel('Total_Power_(W)');
123 xlabel('Time_(hours)');
124 title('Total_Control_Effort');
125 grid on;
126
127 subplot(3, 2, 6);
128 tracking_errors = x_true' - x_bar';
129 plot(t_hours, tracking_errors, 'LineWidth', 1.5);
130 ylabel('Tracking_Error_(C)');
131 xlabel('Time_(hours)');
132 title('True_System_Tracking_Performance');
133 legend('Room_1', 'Room_2', 'Room_3', 'Room_4', 'Room_5', 'Location', 'best
134 grid on;

```

1.3 Matlab code for exc 6

```

1 clear all; close all; clc;
2
3 A = [9.9985 * 10^-1, 9.8510 * 10^-3;
4      -2.9553 * 10^-2, 9.7030 * 10^-1];
5 B = [4.9502 * 10^-5;
6      9.8510 * 10^-3];
7
8 H = [1, 0];
9
10 Q = 1;
11 R = 0.01;
12 N = 1001;
13 dt = 0.01;
14 time = (0:N-1)' * dt;
15
16 x0_true = [1; 1];
17
18 rng(42)
19 w = sqrt(Q) * randn(N,1);
20 v = sqrt(R) * randn(N,1);
21 x_true = zeros(2, N);

```

```

22     x_true(:,1) = x0_true;
23
24     for k = 1:N-1
25         x_true(:,k+1) = A * x_true(:,k) + B * w(k);
26     end
27
28     y_meas = H * x_true + v';
29     function [x_hat_plus, P_plus, x_hat_minus, P_minus] = ...
30         runKalmanFilter(A, B, H, Q, R, y_meas, x_hat0_minus, P0_minus, N)
31         x_hat_minus = zeros(2, N);
32         x_hat_plus = zeros(2, N);
33         P_minus = zeros(2,2,N);
34         P_plus = zeros(2,2,N);
35
36         x_hat_minus(:,1) = x_hat0_minus;
37         P_minus(:, :, 1) = P0_minus;
38         for k=1:N
39             Kk = P_minus(:, :, k) * H' / (H * P_minus(:, :, k) * H' + R);
40             x_hat_plus(:, k) = x_hat_minus(:, k) + Kk * (y_meas(k) - H * x_hat_minus(:, k));
41             P_plus(:, :, k) = (eye(2) - Kk * H) * P_minus(:, :, k);
42
43             if k < N
44                 x_hat_minus(:, k+1) = A * x_hat_plus(:, k);
45                 P_minus(:, :, k+1) = A * P_plus(:, :, k) * A' + B * Q * B';
46             end
47         end
48     end
49
50     [x_hat_plus_1, P_plus_1] = runKalmanFilter(A, B, H, Q, R, y_meas, [1.5; 0.], P0_minus, N);
51     [x_hat_plus_2, P_plus_2] = runKalmanFilter(A, B, H, Q, R, y_meas, [50; -50], P0_minus, N);
52     [x_hat_plus_3, P_plus_3] = runKalmanFilter(A, B, H, Q, R, y_meas, [2; -1], P0_minus, N);
53     [x_hat_plus_4, P_plus_4] = runKalmanFilter(A, B, H, Q, R, y_meas, [1; 1], P0_minus, N);
54     [x_hat_plus_5, P_plus_5] = runKalmanFilter(A, B, H, Q, R, y_meas, [-20; 30], P0_minus, N);
55
56     error_1 = x_true - x_hat_plus_1;
57     error_2 = x_true - x_hat_plus_2;
58     error_3 = x_true - x_hat_plus_3;
59     error_4 = x_true - x_hat_plus_4;
60
61     dist1 = sqrt(sum(error_1.^2, 1));
62     dist2 = sqrt(sum(error_2.^2, 1));
63     dist3 = sqrt(sum(error_3.^2, 1));
64     dist4 = sqrt(sum(error_4.^2, 1));
65
66     figure;

```

```

67 plot(time, dist1, 'r-', 'LineWidth', 1.5); hold on;
68 plot(time, dist2, 'g-', 'LineWidth', 1.5);
69 plot(time, dist3, 'b-', 'LineWidth', 1.5);
70 plot(time, dist4, 'm-', 'LineWidth', 1.5);
71 xlabel('Time(s)');
72 ylabel('Euclidean Distance');
73 title('Distance Between True and Estimated State - Interesting Scenarios');
74 legend(...
75     'Case1: Slightly Wrong + Overconfident', ...
76     'Case2: Very Wrong + Very Uncertain', ...
77     'Case3: Somewhat Wrong + Realistic', ...
78     'Case4: Perfect + Overconfident', ...
79     'Location', 'best');
80 grid on;
81
82 figure('Position', [80, 80, 900, 1200]);
83
84 subplot(4,2,1);
85 sigma_bounds_1_state1 = 3 * squeeze(sqrt(P_plus_1(1,1,:)));
86 plot(time, error_1(1,:), 'b', 'LineWidth', 1.5); hold on;
87 plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
88 plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
89 ylabel('Error');
90 title('Case1: State 1 Error (x0=[1.5; 0.5], P=[0.0001 0; 0 0.0001])');
91 legend('Error', '3 Bound', 'Location', 'best');
92 grid on;
93
94 subplot(4,2,2);
95 sigma_bounds_1_state2 = 3 * squeeze(sqrt(P_plus_1(2,2,:)));
96 plot(time, error_1(2,:), 'b', 'LineWidth', 1.5); hold on;
97 plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
98 plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
99 ylabel('Error');
100 title('Case1: State 2 Error (x0=[1.5; 0.5], P=[0.0001 0; 0 0.0001])');
101 legend('Error', '3 Bound', 'Location', 'best');
102 grid on;
103
104 subplot(4,2,3);
105 sigma_bounds_2_state1 = 3 * squeeze(sqrt(P_plus_2(1,1,:)));
106 plot(time, error_2(1,:), 'b', 'LineWidth', 1.5); hold on;
107 plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
108 plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
109 ylabel('Error');
110 title('Case2: State 1 Error (x0=[50; -50], P=[100 0; 0 100])');
111 legend('Error', '3 Bound', 'Location', 'best');

```

```

112 grid on;
113
114 subplot(4,2,4);
115 sigma_bounds_2_state2 = 3 * squeeze(sqrt(P_plus_2(2,2,:)));
116 plot(time, error_2(2,:), 'b', 'LineWidth', 1.5); hold on;
117 plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
118 plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
119 ylabel('Error');
120 title('Case_2: State_2 Error (x0=[50;-50], P=[100 0; 0 100])');
121 legend('Error', '3 Bound', 'Location', 'best');
122 grid on;
123
124 subplot(4,2,5);
125 sigma_bounds_3_state1 = 3 * squeeze(sqrt(P_plus_3(1,1,:)));
126 plot(time, error_3(1,:), 'b', 'LineWidth', 1.5); hold on;
127 plot(time, sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
128 plot(time, -sigma_bounds_1_state1, 'r--', 'LineWidth', 1);
129 ylabel('Error');
130 title('Case_3: State_1 Error (x0=[2;-1], P=[0.5 0.1; 0.1 0.5])');
131 legend('Error', '3 Bound', 'Location', 'best');
132 grid on;
133
134
135 subplot(4,2,6);
136 sigma_bounds_3_state2 = 3 * squeeze(sqrt(P_plus_3(2,2,:)));
137 plot(time, error_3(2,:), 'b', 'LineWidth', 1.5); hold on;
138 plot(time, sigma_bounds_3_state2, 'r--', 'LineWidth', 1);
139 plot(time, -sigma_bounds_3_state2, 'r--', 'LineWidth', 1);
140 ylabel('Error');
141 title('Case_3: State_2 Error (x0=[2;-1], P=[0.5 0.1; 0.1 0.5])');
142 legend('Error', '3 Bound', 'Location', 'best');
143 grid on;
144
145 subplot(4,2,7);
146 sigma_bounds_4_state1 = 3 * squeeze(sqrt(P_plus_4(1,1,:)));
147 plot(time, error_4(1,:), 'b', 'LineWidth', 1.5); hold on;
148 plot(time, sigma_bounds_4_state1, 'r--', 'LineWidth', 1);
149 plot(time, -sigma_bounds_4_state1, 'r--', 'LineWidth', 1);
150 ylabel('Error');
151 title('Case_4: State_1 Error (x0=[1;1], P=[0.001 0; 0 0.001])');
152 legend('Error', '3 Bound', 'Location', 'best');
153 grid on;
154
155 subplot(4,2,8);
156 sigma_bounds_4_state2 = 3 * squeeze(sqrt(P_plus_4(2,2,:)));

```

```
157 plot(time, error_4(2,:), 'b', 'LineWidth', 1.5); hold on;
158 plot(time, sigma_bounds_4_state2, 'r--', 'LineWidth', 1);
159 plot(time, -sigma_bounds_4_state2, 'r--', 'LineWidth', 1);
160 ylabel('Error');
161 title('Case_4: State_2_Error (x0=[1; 1], P=[0.001 0; 0 0.001])');
162 legend('Error', '3 Bound', 'Location', 'best');
163 grid on;
```